מספר תעודת זהות
יש לרשום את כל תשע הספרות

יש להדביק כאן את
מדבקת הנבחן

האוניברסיטה הפתוחה

כ"ז באלול תשפ"ג

13

מס' שאלון - 524

בספטמבר 2023

סמסטר 2023ב

מס' מועד 99

20905 / 4

שאלון בחינת גמר

20905 - שפות תכנות

משך בחינה: 3 שעות

בשאלון זה **8** עמודים

מבנה הבחינה:

בבחינה 3 שאלות.
משך זמן הבחינה 3 שעות.
עליכם לענות על כל השאלות.
משקל השאלות מפורט בגוף השאלון.

שימו לב:
את תשובותיכם רשמו אך ורק במחברת הבחינה.
כתבו בכתב יד ברור ומסודר.

חומר עזר המותר בשימוש:
ספר הקורס "Essentials of Programming Languages".
מדריך הלמידה "שפות תכנות".

חומר עזר:
ספר הקורס: essentials of programming languages
מדריך הלמידה. מותרות הערות בכתב יד, ע"ג הספרים.
אין להכניס חומר מודפס, דיגיטלי או כל חומר אחר מכל סוג שהוא.

בהצלחה !!!

אינכם חייבים
להחזיר את השאלון לאוניברסיטה הפתוחה

שאלה 1 (35 נקודות)

שאלה זו עוסקת בשדרוג של שפת "ויהי" (שפת LET) המתוארת בספר הלימוד בפרק 3 בעמודים 60-74.

בשאלה זו נרצה להוסיף לשפת "ויהי" ביטוי חדש מסוג לולאת do המוגדרת על פי הדקדוק הבא ובאופן המוסבר ומודגם לאחר מכן.

להלן הדקדוק של לולאת do :

Expression ::= do (
{ < ***Identifier*** ***Expression*** ***Expression*** > }$^{+}$
{ [***Expression*** ***Expression***] }$^{+}$

)

```
do-exp (ids inits steps bools results)
```

הסבר לדקדוק:

ביטוי do-exp מגדיר כמות שרירותית (לפחות אחד) של משתנים מספריים (ids), למשתנים יש ערכים התחלתיים הנתונים ע"י ערכם של הביטויים inits, ובכל מחזור לולאה המשתנים גדלים בצעדים הנתונים ע"י ערכם של הביטויים steps.

כמו כן, ללולאה כמות שרירותית (לפחות אחד) של ביטויים בוליאניים bools, בכל מחזור לולאה נבדקים הביטויים הבוליאניים, והראשון מביניהם שערכו אמת, יגרום ללולאה להיעצר ולהחזיר את ערכו המתאים של הביטוי מתוך results המתאים לביטוי הבוליאני שנמצא שערכו אמת.

אם בבדיקת הביטויים הבוליאניים אין ביטוי שערכו אמת, הלולאה תעבור למחזור הבא שלה, כאשר כל אחד מהמשתנים גדל על פי הצעד המתאים לו, ואז שוב נבדקים הביטויים הבוליאניים כמו קודם. הלולאה תתבצע שוב ושוב, עד אשר ימצא מחזור לולאה שבו אחד התנאים הבוליאניים היה אמת. (כמובן שיכולה להתרחש לולאה אינסופית אך זו אחריותו של המתכנת ולא של המפרש)

הבהרות:

- אתחול של משתנה לולאה do יכול להכיר כבר את המשתנים שכבר אותחלו לפניו.
- קידום משתני לולאה נעשה על פי ערכם של המשתנים במצב הנוכחי

בעמוד הבא מופיעות מספר דוגמאות להמחשת אופן הפעולה של לולאת do

דוגמה 1:

```
> (run "do(
            <a 0 4>
            <b 7 -(a,2)>
            <c -(b,1)  -(a,b)>
            [zero? (b)       -(a,c)]
            [zero? (-(c,-2))  -(a,5)]
        )

")
(num-val 3)
>
```

בדוגמה זו, התוכנית מורכבת מלולאת do המגדירה 3 משתנים a,b,c. בתחילה a שווה לאפס, b שווה ל- 7 , c שווה ל -6 (b פחות 1).

כעת נבדקים התנאים הבוליאנים לפי סדר הופעתם. התנאי הבוליאני הראשון בודק האם ערכו של b הוא 0, תנאי זה אינו מתקיים כרגע. התנאי הבוליאני השני בודק האם ערכו של c הוא 2-, גם הוא לא מתקיים כרגע.

מאחר ואף תנאי בוליאני לא התקיים, הלולאה ממשיכה למחזור הבא שלה, שבו היא תעבוד עם משתנים a,b,c שערכם הוא כעת:

a שווה ל-4, כי היה לו 0 והתווסף לו 4

b שווה ל-5, כי היה לו 7 והוא גדל ב-a-2, a במצבו הנוכחי (לפני שקודם) היה 0 ולכן a-2 זה 2-

ולכן b קטן ב-2 מ- 7 ל-5

c שווה ל- 1-, כי היה לו 6 והתווסף לו a-b, ערכם של a ו-b (לפני שקודמו) היה 0 ו-7 בהתאמה ולכן

c קטן ב-7- ונהיה 1-

כעת נבדקים שוב התנאים הבוליאניים, וניתן לראות ששוב אף אחד מהם אינו נכון, ולכן הלולאה ממשיכה למחזור הבא שלה, עם משתנים a,b,c שערכם הוא:

a שווה ל-8, כי היה לו 4 והתווסף לו 4.

b שווה ל-7, כי היה לו 5, והוא גדל ב- a-2 שזה 2

c שווה ל-2-, כי היה לו 1- והוא גדל ב- a-b שזה 1-

וכעת שוב נבדקים התנאים הבולאניים (לפי הסדר בו הם כתובים) וכעת התנאי הבוליאני השני מתקיים! ולכן הלולאה נעצרת ומוחזר ערכו של הביטוי הכתוב לצד התנאי הבוליאני שערכו היה אמת, שבמקרה זה זה הביטוי a-5 שערכו עלפי ה- a הנוכחי הוא 8-5=3 ולכן תוצאת התוכנית היא 3.

דוגמה 2:

```
> (run "do(

            [zero? (b)        -(a,c)]
            [zero? (-(c,-2))   -(a,5)]
        )

")
interp.scm:72:15: do-exp: do loop with no variables are not allowed
> |
```

בדוגמה זו יש לולאת do ללא הגדרת משתנים, על פי הדקדוק הנתון בשאלה נדרש לפחות משתנה אחד ולכן התקבלה הודעת שגיאה מתאימה.

דוגמה 3:

```
> (run "do(
            <a 0 4>
            <b 7 -(a,2)>
            <c -(b,1)   -(a,b)>

        )

")
interp.scm:74:19: do-exp: do loop with no booleans are not allowed
> |
```

בדוגמה זו יש לולאת do ללא הגדרת תנאים בוליאנים, על פי הדקדוק הנתון בשאלה נדרש לפחות תנאי בוליאני אחד ולכן התקבלה הודעת שגיאה מתאימה.

ממשו והטמיעו את השינויים הדרושים כפי שהוגדרו והודגמו בתוך שפת **"ויהי"** (**שפת LET**).
בתשובתכם, הקפידו להסביר **היכן בדיוק** נדרשים שינויים ותוספות בקבצי המפרש, **ומהם** השינויים והתוספות.

הנחיות כלליות לפתרון:

1. הקפידו על כתב ברור, ועל קוד מבני ומסודר.
2. הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
3. הפתרון אמור לשמור על הגדרות השפה המקוריות (פרט למקרים בהם נדרש שינוי כזה במפורש בשאלה)
4. על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
5. בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
6. ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב- terminals, non-terminals ופעולות של סגור ($\{\ \}^*$ או $\{\ \}^+$)

שאלה 2 (35 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת **"וירמוז"**(IMPLICIT-REFS).

בשאלה זו, נרצה לאפשר הוספת הגנה אופציונלית למשתנים מפני ניסיון להשמת ערך או עדכון ערך קיים הגורם להפרה של ההגנה על המשתנה.

למשל אם הוגדר משתנה בשם w והוצהרה עבורו הגנה עד כך שהוא אמור לקבל תמיד ערך מספרי, אזי אם יהיה ניסיון לתת לו ערך שאינו מספרי (הפרה של ההגנה עליו) אזי תודפס הודעת שגיאה מתאימה וריצת התוכנית תיעצר.

אם משתנה הוגדר ללא הגנה עליו כלל, אזי ניתן יהיה לשנותו לכל סוג ערך שנחפוץ, למשל אם ערכו כרגע מספרי, ניתן יהיה לשנותו למשל להיות פרוצדורה או ערך בוליאני.

להזכירכם, בשפת "וירמוז" משתנים מוחזקים בסביבה וערכם נמצא בהחסן, ומהסביבה ישנו reference למקום בהחסן שם נמצא הערך של המשתנה.

בשפת "וירמוז" ישנה עבודה עם משתנים ומתן ערכים עבורם במסגרת הביטויים הבאים:

- בביטוי let בו מגדירים משתנה ונותנים לו ערך.
- בהגדרת פרוצדורה לה יש פרמטר, וכאשר מפעילים את הפרוצדורה שולחים לפרמטר ערך.
- בביטוי set-exp בו רוצים לשנות למשתנה קיים את ערכו הנוכחי לערך אחר.

לשם מתן אפשרות למתכנת להציב הגנה אופציונלית על משתנים, נשנה את התחביר של ביטוי let-exp ושל ביטוי להגדרת פרוצדורה proc-exp לפי הדקדוקים הבאים:

להלן הדקדוקים החדשים המגדירים מחדש את ביטויי let-exp ו-proc-exp:

Expression ::= let ***Guard Identifier*** = ***Expression***
in ***Expression***

```
let-exp (grd id exp1 body)
```

Expression ::= proc (***Guard Identifier***) ***Expression***

```
proc-exp (grd var body)
```

Guard ::= ε | # | ? | @

בעמוד הבא מופיע הסבר על הדקדוק

הסבר התחביר:

Guard - מוגדר להיות # המציין שהמשתנה אמור להיות תמיד מספרי במהלך ריצת התוכנית.

או ? המציין שהמשתנה אמור להיות תמיד בעל ערך בוליאני לאורך ריצת התוכנית.

או @ המציין שהמשתנה אמור להיות תמיד מסוג פרוצדורה(כלשהי) לאורך ריצת התוכנית.

או המילה הריקה (פשוט כלום – זה מה שמייצג ε) ובמקרה כזה למשתנה אין שום הגנה.

ביטוי let-exp החדש - מאפשר להגדיר משתנה עבור ה-let עם אפשרות להגנה עליו.

ביטוי proc-exp החדש – מאפשר להוסיף הגנה אופציונלית על שם הפרמטר של הפרוצדורה.

להלן מספר דוגמאות להמחשת העבודה עם משתנים מוגנים ולא מוגנים :

דוגמה 1:

```
> (run "let # q = zero?(9)
        in q")
interp.scm:74:16: guard-violation: expression #(struct:zero?-exp
#(struct:const-exp 9)) violate let guard
>
```

בדוגמה זו מוגדר משתנה בשם q עם הגנה על ערך מספרי. מאחר ומנסים להציב לו ערך בוליאני המפר את ההגנה עליו, הודפסה הודעת שגיאה מתאימה.

דוגמה 2:

```
> (run "let @ q=proc (# w) -(w,1)
       in
        (q zero? (9))")
interp.scm:151:14: guard-violation: argument violate procedure bound variable w guard
>
```

בדוגמה זו, מוגדר משתנה q עם הגנה על ערך מסוג פרוצדורה. ל- q אכן ניתנת פרוצדורה כערך.

לפרוצדורה יש פרמטר w עם הגנה על ערך מספרי, בגוף ביטוי ה-let נעשית קריאה לפרוצדורה עם ארגומט עבור w שאינו מספרי המפר את ההגנה על w ולכן התקבלה שגיאה.

דוגמה 3:

```
> (run "let # w = 8
        in
          set w = zero?(9)")
interp.scm:104:15: guard-violation: assign expression violate variable w guard
>
```

בדוגמה זו, מוגדר משתנה w עם הגנה על ערך מספרי, המשתנה אכן מקבל ערך מספרי של 8.
לאחר מכן, יש ניסיון לשנותו לערך בוליאני המפר את ההגנה עליו, ולכן התקבלה שגיאה.

דוגמה 4:

```
> (run "let w = 8
        in
         begin
          set w = zero?(9);
          if w then 1 else 2
         end")
(num-val 2)
>
```

דוגמה זו ממחישה הגדרת משתנה בשם w ללא שום הגנה, בתחילה ניתן למשתנה ערך מספרי של 8.

לאחר מכן, הוצב במשתנה ערך מסוג אחר (בוליאני) - ואין כאן שום הפרה מאחר ואין שום הגנה.

ממשו והטמיעו את השינויים הדרושים בתוך שפת **"וירמוז"** (**שפת IMPLICIT-REFS**) לתמיכה בהתנהגות החדשה הדרושה בעבודה מול משתנים.

ייתכנו מספר כיווני פתרון אפשריים. בתשובתכם, הקפידו להסביר **היכן בדיוק** נדרשים שינויים ותוספות במרכיבים השונים של המפרש, **ומהם** השינויים והתוספות.

הנחיות כלליות לפתרון:

1. הקפידו על כתב ברור, ועל קוד מבני ומסודר.
2. הקפידו על פתיחת וסגירת סוגריים במקומות הנכונים
3. הפתרון אמור לשמור על הגדרות השפה המקוריות. (כגון אופן ניהול הסביבה וסוגי הערכים בה, אופן ניהול ההחסן והערכים שניתן לשמור בו, וכדומה)
4. על מנת לפשט קוד ארוך, כדאי ורצוי להיעזר בכתיבת פרוצדורות עזר (מחוץ ל-value-of או כאלה המוטמעות בתוכו)
5. בפתרונכם, הקפידו על אבחנה ושימוש נכון בין ביטוי (expression) לבין תוצאת חישוב ביטוי (expval) לבין identifier
6. ניקוד יורד על אי ניתוח ומימוש נכון של הדקדוק הנתון בשאלה, כלומר יש להפקיד על זיהוי וטיפול נכון ב- terminals, non-terminals ופעולות של סגור ($\{\}^*$ או $\{\}^+$)

שאלה 3 (30 נקודות)

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
let p=proc(x: ?) -(x,5)
in
  (p -((p 5),5))
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים).לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (15 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש ב- substitution ו- unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

ב ה צ ל ח ה !